TURBOCHARGING PARALLEL BACKTESTING WITH HIGH-PERFORMANCE STORAGE SYSTEMS

Lan Nguyen, **Ioan Raicu**

lnguyen18@hawk.illinoistech.edu, iraicu@illinoistech.edu
Illinois Institute of Technology

ILLINOIS TECH
College of Computing

DataSys
Data-Intensive Distributed Systems Laboratory

ABSTRACT

The rapid growth of high-frequency time-series data across domains such as computational finance and scientific simulation has introduced significant challenges in storage, querying, and large-scale analysis. Workloads such as parameter sweeps in backtesting pipelines often require executing billions of simulations over decades of fine-grained data, placing extreme pressure on storage systems in terms of throughput, scalability, and resource utilization. Traditional storage solutions such as MongoDB, InfluxDB, CSV (via Pandas), and Parquet experience difficulty with balancing performance and efficiency under highly concurrent workloads and diverse access patterns.

MOTIVATIONS

- Architectural Strain: Conventional storage systems struggle to maintain performance as time-series data expands toward the petabyte horizon
- Parallelism: Billions of simulations create highly concurrent, bursty access
- Data Access: Redundant scans and poor locality increase overhead
- I/O Bottleneck: Storage throughput and latency dominate backtesting runtime
- Resource overhead: Sustained performance requires intensive CPU, memory, and I/O utilization

TESTBED

	Nodes	Sockets	Processor	Memory	Disk	Network
Client	8	2	Intel SP 4108 16C @1.8GHz	64GB	×4 NVMe SSD (RAID-0)	100 GbE
Server	1	2	Intel SP 4108 16C @1.8GHz	64GB	×4 NVMe SSD (RAID-0)	100 GbE

DATASETS

Table 1: Micro-benchmark Dataset

Timestamp	Col0	Col1	...	Col127
946684800	-319,156.3	-626,002.9	...	-762,068.4
946684801	-850,582.6	-639,529.9	...	748,292.6
...	...	...	...	...
1262303998	610,269.2	93,676.0	...	529,575.3
1262303999	630,937.5	-593,804.7	...	457,526.3

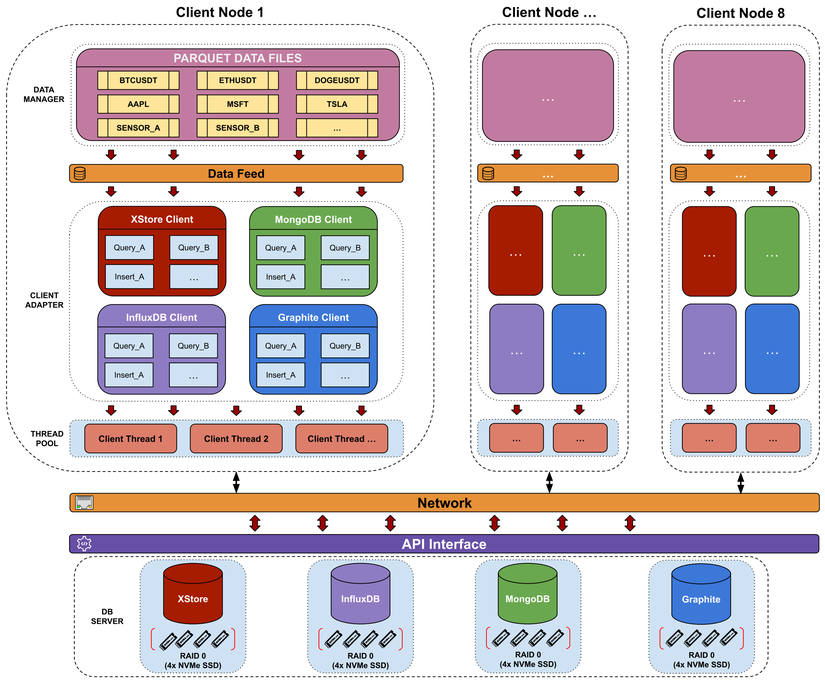
Table 2: Backtest Dataset

Timestamp	Open	High	Low	Close	Volume
946684800	100.0	100.4	99.5	100.0	762,463
946684801	100.0	100.2	98.7	99.0	564,932
...	...	...	...	...	...
1262303998	38,847.8	38,849.5	38,844.9	38,846.5	127,615
1262303999	38,846.5	38,847.6	38,846.1	38,847.2	113,092

MICRO-BENCHMARK DESIGN & IMPLEMENTATION

The workflow of the benchmark program consists of four steps:

1. Load pre-generated data into memory
2. Initialize client adapter with the loaded dataset
3. Assemble a pool of clients to a ready state
4. Dispatch workloads according to the poller's schedule



BACKTESTING

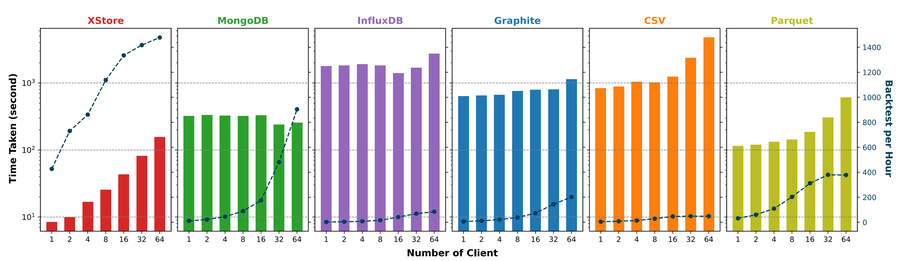
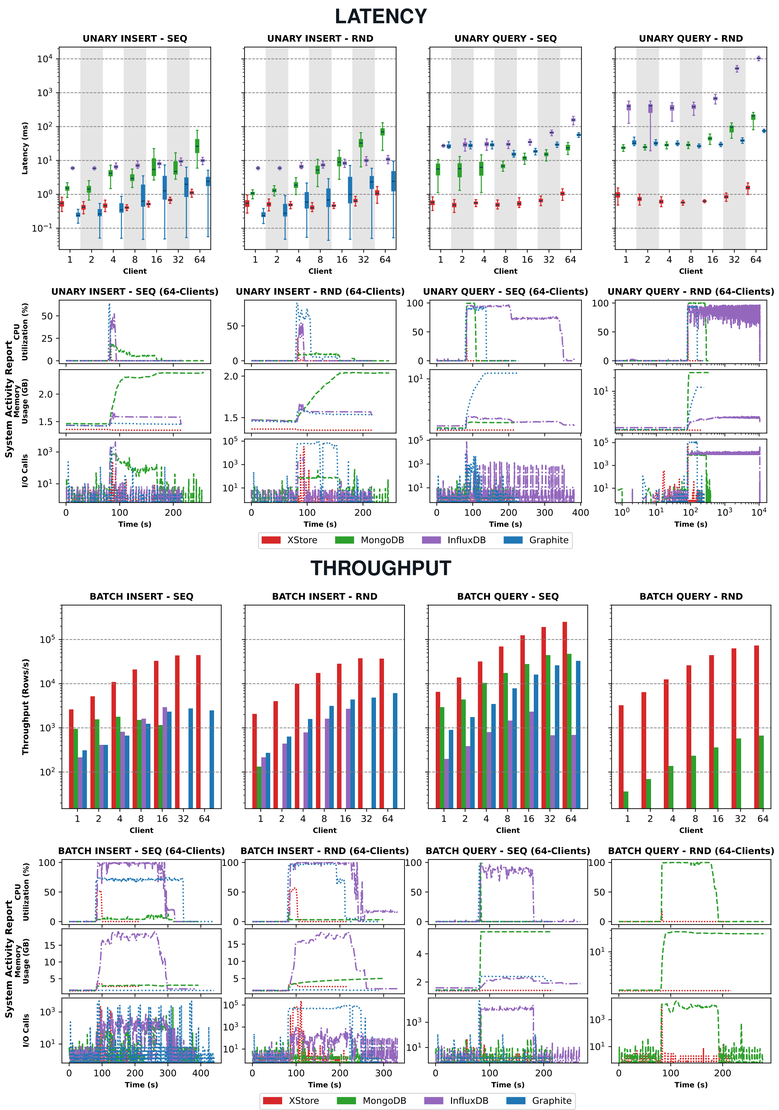


Figure: Backtesting of financial model for hyper-parameter tuning: Bars (L-axis) vs. Lines (R-axis)

MICRO-BENCHMARKS



CONCLUSION

- Explosive time-series growth requires more efficient storage and querying
- Indexless, row-optimized architecture removes trees and heavy caching
- Direct timestamp-to-offset computation eliminates costly index traversal
- Supports long-range, high-resolution time-series queries
- High performance under concurrency without memory-heavy buffering